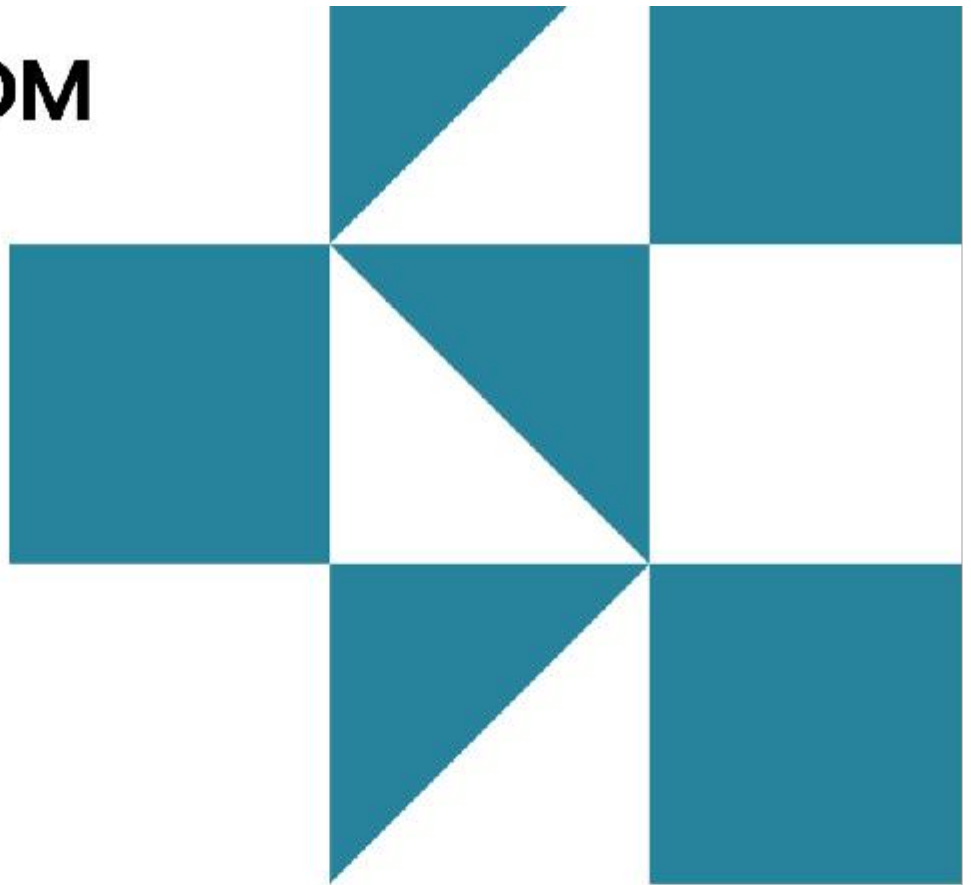




한글로젠

사용자 매뉴얼

02-865-1644

경기 성남시 분당구 대왕판교로644번길 49
한컴타워 2층

www.hancomins.com

목차

I. 솔루션 소개	5
1. 「한글로젠」 개요	5
1) 한글로젠의 구성	6
2. 주요 기능 및 특징	7
1) HWP 파일 제어	7
2) 한글 문서 데이터 연계	8
3) 한글 문서 내용 추출	8
4) 한글 워드프로세서 속성을 갖는 HWP 파일 생성	9
5) 빠른 속도와 대용량 처리	9
II. 시작하기	12
1. 한글로젠 설치 및 초기 설정	12
1) 한글로젠 서버 설치	12
IV. 주요 기능 설명	14
1. 한글 파일 제어	14
1) HWP 파일 읽기	14
2) HWP 파일 저장	15
3) HWP 파일 결합하기	17
2. 필드에서 데이터 가져오기	20
1) 필드	20
2) 조건에 맞는 필드 가져오기	23
3. 필드에 정보 설정하기	25
4. 표 정보 가져오기	27
1) 표 이름 설정하기	27
2) 표 전체의 필드 이름 가져오기	29

3) 셀의 필드 이름 가져오기.....	30
5. 표에 정보 설정하기.....	31
1) CSV 데이터.....	31
2) CSV 데이터 삽입하기.....	32
3) JSON 데이터 삽입하기.....	35
4) 셀 설정하기.....	39
6. API 리스트.....	41
1) HWPGenLoader 클래스.....	41
2) HWPGenSaver 클래스.....	42
3) HWPGenInserter 클래스.....	43
4) HWPGenFieldGetter 클래스.....	43
5) FieldItem 클래스.....	44
6) FieldFindOption 클래스.....	45
7) HWPGenFieldSetter 클래스.....	47
8) HWPGenFieldSetter 클래스.....	47
9) HWPGenTableSetter 클래스.....	49
10) HWPGenTableSetter 클래스.....	50
11) JSONDataSettingInfo 클래스.....	51
V. 문제 해결 및 지원.....	54
1. 자주 묻는 질문(FAQ).....	54



I. 솔루션 소개



I. 솔루션 소개

1. 「한글로젠」 개요

한글로젠은 (주)한컴이노스트림에서 개발한 한글(HWP) 문서 전용 리포팅 솔루션입니다. 본 솔루션은 (주)한글과컴퓨터의 한글 워드프로세서 파일 포맷인 HWP 파일을 생성 및 분석할 수 있는 자바(JAVA) 라이브러리(Library)입니다.

일반적인 리포팅 툴은 객체와 컨트롤의 위치를 x, y 좌표 방식으로 결정하지만, 한글 워드프로세서는 문자를 기준으로 위치를 결정합니다. 이로 인해 리포팅 툴로 변환한 결과를 한글 워드프로세서와 완벽하게 호환하는데 어려움이 있었습니다.



그림 1-1. 리포팅 툴과 워드 프로세서의 표현 방식 비교

한글로젠은 이러한 한계를 극복하기 위해 개발되었습니다. 한글로젠은 한글(HWP) 파일에 데이터를 쉽게 삽입하여 동적으로 HWP 파일을 생성할 수 있는 도구입니다. 이를 통해 신속하고 명확하게 데이터를 연계하여 HWP 파일을 만들 수 있습니다. 또한, 한글 워드프로세서가 없어도 HWP 파일을 제어할 수 있으며, 원본 문서의 기능을 그대로 유지하면서 다양한 방식으로 활용할 수 있습니다.

사용자는 익숙한 한글 환경에서 편리하게 리포팅 작업을 수행할 수 있으며, JSON, CSV 등 다양한 데이터 소스와 연동하여 동적인 문서를 생성할 수 있습니다.

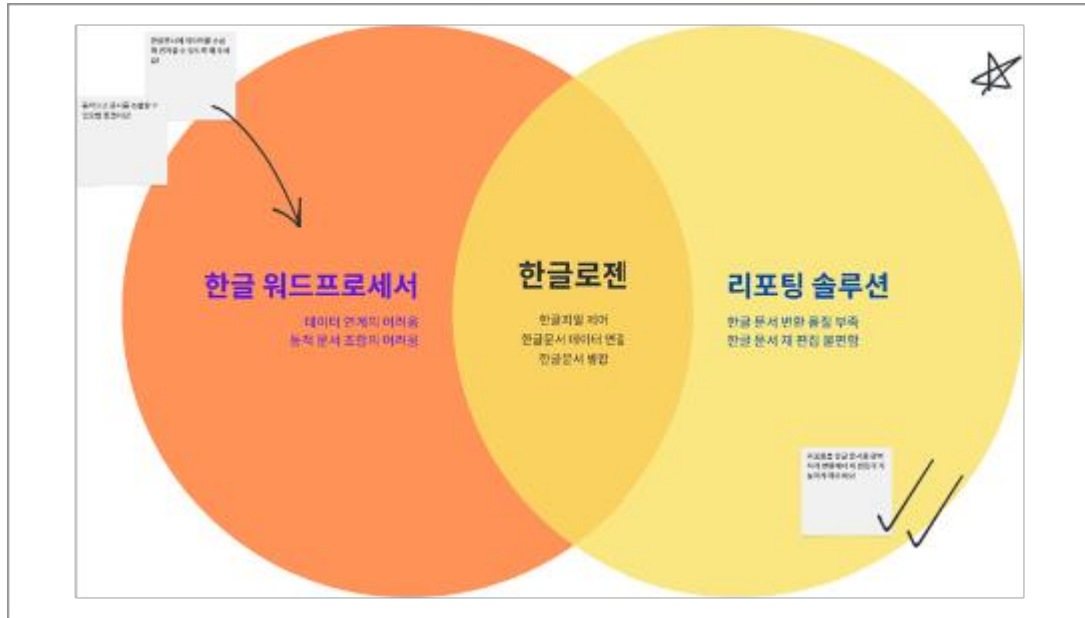


그림 1-2. 한글로젠 개념도

1) 한글로젠의 구성

한글로젠은 JAVA 라이브러리와 디자인어로 구성되어 있습니다. 라이브러리는 3개의 jar파일로 구성되며, 디자인어는 WAS에 설치할 수 있는 war 파일 형식으로 제공합니다.



그림 1-3. 한글로젠 라이브러리

■ HWPLib.jar

HWPLib.jar 라이브러리는 HWP 파일을 읽고 쓰는 등의 기본적인 기능을 지원합니다. 이 라이브러리는 2016년 처음 오픈 소스 형태로 개발하여 현재까지 지속적으로 패치 및 업데이트하고 있습니다.

■ HWPGen.jar

HWPGen.jar 라이브러리는 HWPLib.jar 기반의 리포트 구동방식으로 HWP 파일을 생성하는 기능을 지원합니다. 본 라이브러리는 HWPLib.jar를 직접 사용하기 어려운 사용자를 위해 쉬운 방식으로 HWP 파일을 생성할 수 있는 기능을 제공합니다. 해당 기능을 사용하여 사용자는 쉽고 빠르게 HWP 파일을 생성 및 관리할 수 있습니다.

■ HWPGen_ext.jar

HWPGen_ext.jar 라이브러리는 JSON 데이터 읽기 기능 구현을 위해 필요한 외부 라이브러리의 모음입니다.

Group Id	Artifact Id	Version	License
org.json	Json	20210307	JSON License
com.googlecode.json-simple	json-simple	1.1.1	Apache 2.0
net.minidev	json-smart	2.4.7	Apache 2.0

표 1-1. 외부 라이브러리 모음

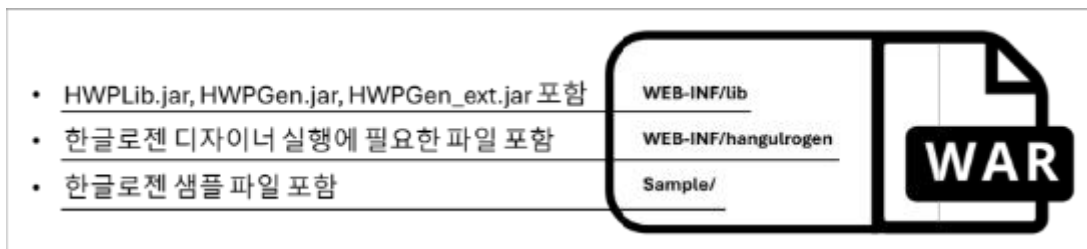


그림 1-4. 한글로젠 디자이너 WAR File

2. 주요 기능 및 특징

1) HWP 파일 제어

한글로젠은 한글 워드프로세서가 설치되지 않는 환경에서도 HWP 파일을 제어할 수 있습니다. 개발자는 JAVA 개발 환경의 한글로젠을 활용하여 HWP 파일을 읽고 JAVA 객체로 변환하여 자유롭게 내용과 속성을 변경할 수 있습니다.



그림 1-6. 한글로젠 HWP 문서 내용 및 속성 수정 예시

2) 한글 문서 데이터 연계

한글로젠은 한글 문서와 데이터(json, csv)를 연계하여 새로운 결과 문서를 생성합니다.



그림 1-7. 한글 문서 데이터 연계

3) 한글 문서 내용 추출

한글로젠은 HWP 파일 내의 특정 부분의 내용을 추출할 수 있습니다. 한글 워드프로세서는 '필드' 기능을 사용하여 문서 내 주요 내용을 표현하거나, 사용자 입력을 받습니다. 한글로젠은 이 '필드' 개체에 접근하여 내용을 가져오는 기능을 지원합니다.

● 사건기본정보	
사건명	가등기말소
청구구분	재산권청구
소가구분	토지 등의 평가액
유형	해고무효확인의 소를 제외한 회사 등 관계소송, 무체재산권에 관한 소송
소가	123,123,123 원
제출법원	춘천지방법원 강릉지원

[추출결과]
 사건명, Cell, 가등기말소
 청구구분, Cell, 재산권청구
 소가구분, Cell, 토지 등의 평가액
 유형, Cell, 해고무효확인의 소를 제외한 회사 등 관계소송, 무체재산권에 관한 소송
 소가, Cell, 123,123,123 원
 제출법원, Cell, 춘천지방법원 강릉지원

표 1-2. 한글 문서의 데이터 추출 예시

4) 한글 워드프로세서 속성을 갖는 HWP 파일 생성

한글로젠을 사용하면 한글 워드프로세서의 속성을 그대로 갖는 HWP 파일을 생성할 수 있습니다. 리포팅 툴의 HWP 변환 과정을 통해 생성된 HWP 파일은 리포팅 툴의 한계가 반영됩니다. 리포팅 툴은 일반적으로 좌표를 기준으로 위치를 정하기 때문에 특정 좌표 기준으로 내용을 표현한 HWP 파일을 생성합니다.



그림 1-8. 리포팅 툴에서 저장한 HWP 파일

이러한 방식은 단순히 내용을 출력할 때에는 문제가 없지만, 문서 생성 이후 재 편집 과정에서 어려움을 겪게 됩니다. 한글로젠은 한글 문서의 속성을 동일하게 적용하여 마치 한글 워드프로세서로 제작한 것과 같이 원활한 편집이 가능한 HWP 파일을 생성할 수 있습니다.



그림 1-9. 한글로젠으로 생성한 HWP 파일

5) 빠른 속도와 대용량 처리

한글과 컴퓨터에서 제공하는 클라이언트 측 API는 한글 문서의 속성이 변경될 때마다 해당 부분을 다시 렌더링 해야 합니다. 이로 인하여 문서 속성 변경 시간과 렌더링 시간이 더해져

전체 처리 시간이 증가합니다. 반면, 한글로젠은 문서의 특정 속성을 변경해도 그것을 다시 그리지 않기 때문에 상대적으로 속도가 빠릅니다.

또한, 클라이언트 측 API는 웹 브라우저와 같은 클라이언트 환경에서 동작하므로 메모리 사용이 제한적입니다. 이로 인해 대용량 문서를 생성하거나 편집하기 어려울 수 있습니다.

한글로젠은 서버 환경에서 동작하므로 메모리를 보다 자유롭게 사용할 수 있습니다. 따라서 대용량 문서 처리에 더욱 효과적입니다.



II. 시작하기



II. 시작하기

1. 한글로젠 설치 및 초기 설정

1) 한글로젠 서버 설치

한글로젠을 사용하기 위해서는 JAVA 프로젝트에 hwplib.jar, hwpngen.jar, hwpngen_ext.jar를 프로젝트에 추가해야 합니다. MAVEN 프로젝트 경우 pom.xml 파일을 아래와 같이 의존성 설정을 추가합니다.

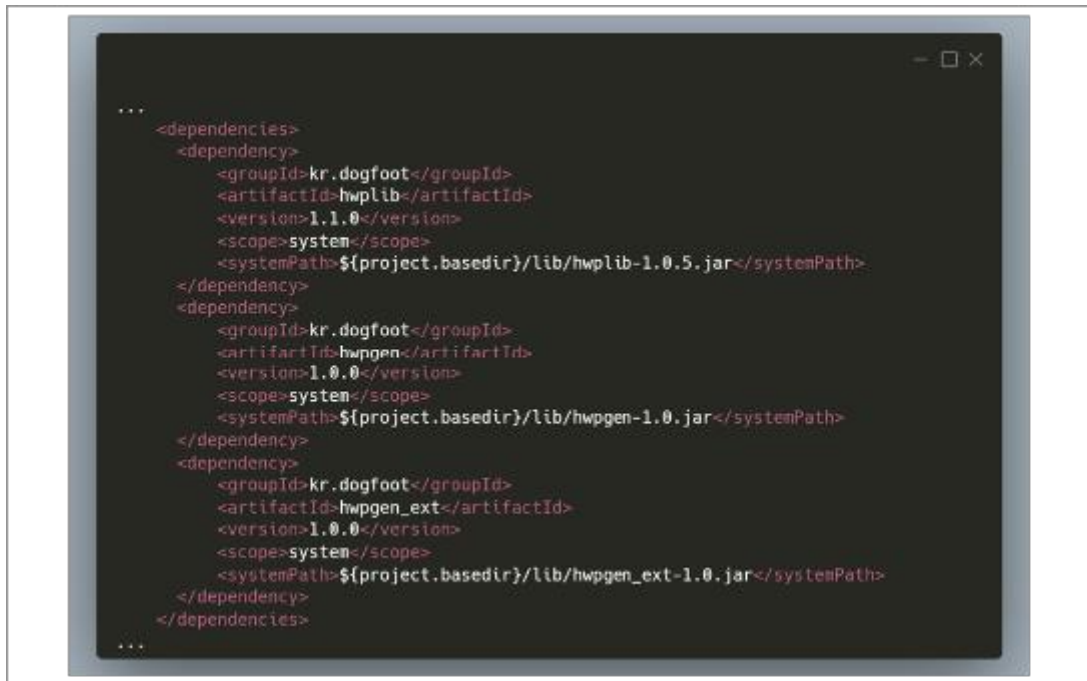


그림 2-1. pom.xml 의존성 설정

MAVEN과 같은 프로젝트 관리 도구를 사용하지 않는다면 한글로젠에서 제공한 jar 파일을 프로젝트에 직접 추가하여 사용할 수 있습니다.

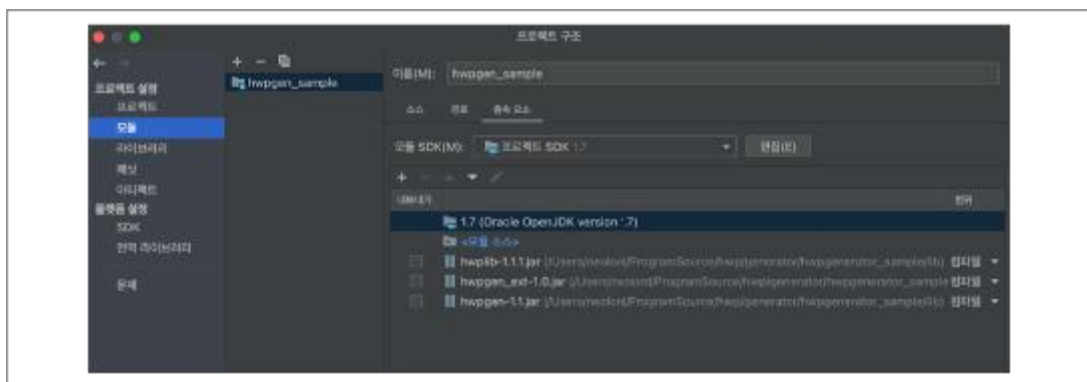


그림 2-2. IntelliJ에서 jar 파일 직접 등록



IV. 주요 기능 설명



IV. 주요 기능 설명

1. 한글 파일 제어

1) HWP 파일 읽기

한글로젠은 `kr.dogfoot.hwpgenerator.smart.HWPGenLoader` 클래스를 통해 HWP 파일을 읽는 다양한 방식을 제공합니다.

■ 파일 경로를 통해 읽기



그림 4-1. 파일경로 사용 예시

특정 경로에 위치한 HWP 파일을 읽기 위해서는 `HWPGenLoader` 클래스의 `fromFilepath()` 함수를 사용합니다. 이 함수는 HWP 파일을 읽고 내용을 `HWPFile` 클래스 타입의 객체로 반환합니다. 한글로젠은 `HWPFile` 객체를 함수에 매개변수로 넘겨줌으로써 원하는 HWP 파일에 접근할 수 있습니다. `HWPFile` 클래스는 HWP 파일을 제어하는 기능을 제공합니다. 이 클래스는 "hwplib.jar" 안에 포함되어 있습니다.

■ URL을 통해 읽기

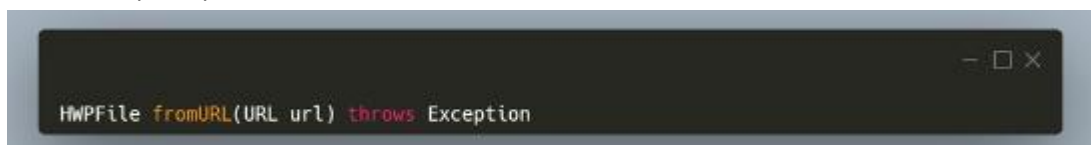


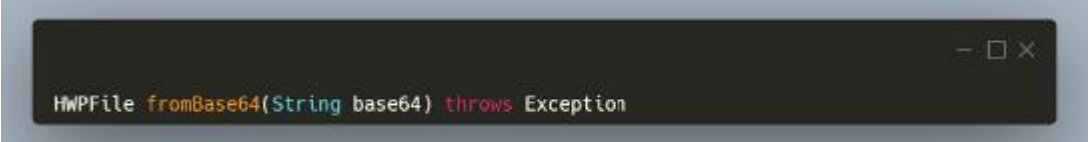
그림 4-2. URL 사용 예시

URL을 사용하여 HWP 파일을 읽기 위해서는 `fromURL()` 함수를 사용합니다. 이 함수를 사용하면 인터넷에 존재하는 HWP 파일을 다운로드하여 읽을 수 있습니다.

■ 스트림(Stream)으로 읽기

JAVA는 `InputStream` 클래스에서 파생한 다양한 클래스를 제공합니다. 파일 데이터 또는, 네트워크 소켓을 통해 데이터를 읽거나 키보드에서 입력한 데이터를 읽을 때 사용합니다.

■ Base64 문자열에서 읽기



```
HWPFile fromBase64(String base64) throws Exception
```

그림 4-3. fromBase64() 함수 예시

Base64 문자열로 저장된 HWP 파일을 읽기 위해 fromBase64() 함수를 사용합니다. 본 함수는 Static 함수로 정의 되어있으며, 아래 예제와 같이 "HWPGenLoader" 클래스를 통해 호출할 수 있습니다.



```
HWPFile hwpFile = HWPGenLoader.fromFilepath("test.hwp");
String base64 = HWPGenSaver.toBase64(hwpFile);
HWPFile hwpFile2 = HWPGenLoader.fromBase64(base64);
```

그림 4-4. Base64 문자열에서 HWP 파일 읽기

※ 유의사항

대용량 HWP 파일을 읽는 경우 메모리 부족 등의 이유로 오류가 발생할 수 있습니다. 오류가 발생하는 경우 JRE의 메모리 관련 옵션을 재설정하거나, 하드웨어적으로 증설하는 조치를 통해 방지할 수 있습니다.

2) HWP 파일 저장

한글로젠을 사용하여 필드나 표에 정보를 설정하면 HWP 파일 객체가 변경됩니다. 변경된 HWP 파일 객체를 파일로 저장하기 위해서는 "HWPGenSaver" 클래스에 포함된 다음과 같은 함수들을 사용합니다.

■ 파일로 저장하기



```
void toFilepath(HWPFile hwpFile, String filepath) throws Exception
```

그림 4-5. toFilepath() 함수 사용 예시

특정 경로에 HWP 파일을 저장하기 위해서는 HWPGenSaver 클래스의 toFilepath() 함수를 사용합니다. 이 함수는 HWP 파일 객체와 경로를 매개변수로 받습니다. HWPGenSaver 클래스에서 제공하는 모든 함수는 HWP 파일을 저장하는 과정에서 발생하는 오류에 대해 Exception을 반환하여 이를 통해 에러 처리가 가능하도록 돕습니다.

■ OutputStream을 통해 저장하기

```
void toOutputStream(HWPFile hwpFile, OutputStream os) throws Exception
```

그림 4-6. toOutputStream() 함수 사용 예시

JAVA는 OutputStream 클래스에서 파생된 다양한 클래스를 제공합니다. 이러한 클래스를 사용하여 다양한 형태의 출력방식이나 출력장치를 사용합니다. JAVA 개발자는 위 함수를 사용하여 다양한 출력방식으로 출력장치에 HWP 파일을 저장할 수 있습니다.

■ Base64 문자열로 저장하기

```
String toBase64(HWPFile hwpFile) throws Exception
```

그림 4-7. toBase64() 함수 사용 예시

toBase64() 함수는 HWP 파일을 Base64 문자열로 변환하여 반환합니다. 이 함수는 Static 함수로 정의되어 아래 예제와 같이 "HWPGenSaver" 클래스를 사용하여 호출할 수 있습니다.

```
HWPFile hwpFile = HWPGenLoader.fromFilepath("settable.hwp");
HWPGenTableSetter.setCellText(hwpFile, "성적테이블", 1, 1, "ABD\\ndEF",
    "https://www.naver.com");
HWPGenTableSetter.setCellFieldName(hwpFile, "성적테이블", 1, 1, "판드1_1");
HWPGenSaver.toFilepath(hwpFile, "settable_result.hwp");
```

그림 4-8. HWP 파일 저장

```
HWPFile hwpFile = HWPGenLoader.fromFilepath("settable.hwp");
String base64 = HWPGenSaver.toBase64(hwpFile);
System.out.println(base64);
```

그림 4-9. HWP 파일을 Base64 문자열로 저장하여 출력 예제

3) HWP 파일 결합하기

한글로젠은 "HWPGenInssterter"클래스를 사용하여 HWP 파일에 다른 HWP 파일을 삽입할 수

있습니다.

■ 본문 끝에 이어 붙이기

본문 끝에 이어서 다른 HWP 파일을 삽입하기 위해 appendToEndOfBody() 함수를 사용합니다.

```
public static void appendToEndOfBody(HWPFile origin,
    HWPFile addition, AppendOption option) throws Exception
```

그림 4-10. appendToEndOfBody() 함수 사용 예제

이 함수는 origin, addition이라는 HWP 파일 객체와 option을 매개변수로 받아, origin 파일의 본문 끝에 파일을 결합합니다.

option 매개변수는 다음과 같은 값을 선택할 수 있습니다.

```
public enum AppendOption {
    FromLastSection,
    FromNewSection,
    FromNewPage
}
```

그림 4-11. option 매개변수

AppendOption.FromLastSection 값은 origin 파일의 마지막 구역에 addition 파일을 추가합니다. 이 옵션은 addition 파일의 내용을 origin 파일의 마지막 구역 안의 내용인 것처럼 추가합니다.



그림 4-12. origin 파일과 addition 파일

예를 들어 origin 파일과 addition 파일이 위와 같을 경우, 아래와 같이 JAVA 코드를 작성하여 실행하면 다음과 같은 결과를 얻을 수 있습니다.

```

HWPFile originFile = HWPGenLoader.fromFilepath("origin.hwp");
HWPFile additionFile = HWPGenLoader.fromFilepath("addition.hwp");

HWPGenInserter.appendToEndOfBody(originFile, additionFile,
    AppendOption.FromLastSection);

HWPGenSaver.toFilepath(hwpFile, "append_result.hwp");

```

그림 4-13. appendToEndOfBody 사용 예시



그림 4-14. appendToEndOfBody 결과 문서

위와 동일한 방식으로, AppendOption.FromNewSection 값을 option 매개변수에 설정하면 다음 결과와 같이 addition 파일의 내용이 새로운 구역에서 시작됩니다.

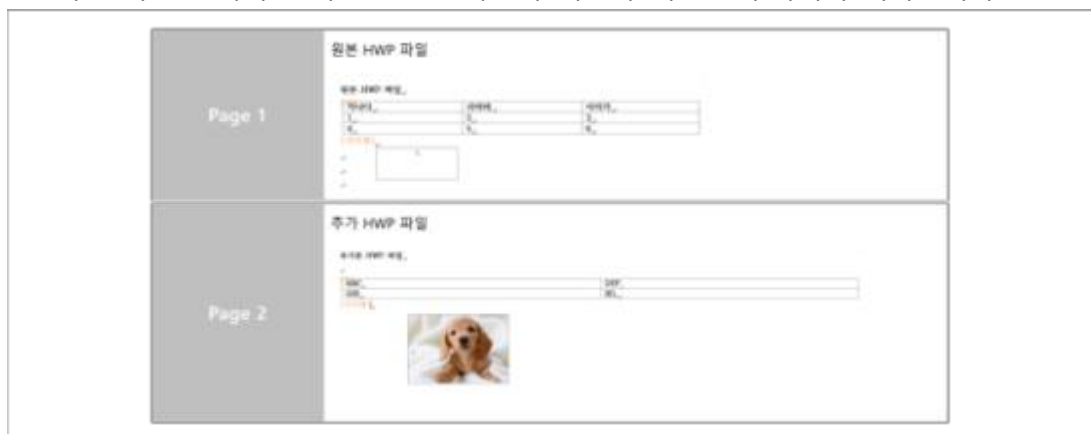


그림 4-15. AppendOption.FromNewSection으로 두 파일을 결합한 결과

그림 27의 결과 HWP 문서는 첫 페이지는 세로 방향, 두 번째 페이지는 가로방향으로 설정되어 있다. 이와 같이 AppendOption.FromNewSection 옵션으로 두 파일을 결합하면 위와 같이 서로 다른 페이지 설정을 갖는 두 개의 구역을 포함하는 HWP 문서를 생성할 수 있습니다.

마지막 옵션인, AppendOption.FromNewPage는 addition 파일 붙이기 전에 origin 파일에 본문 끝에 [페이지 나누기]를 넣어 준 다음, AppendOption.FromLastSection과 동일하게 origin 파일의 마지막 섹션에 addition 파일의 내용을 추가합니다. appendToEndOfBody() 함수의 option 매개변수를 AppendOption.FromNewPage로 설정하고 실행하면 다음과 같은 결과를 얻을 수 있습니다.



그림 4-16. AppendOption.FromNewPage를 사용하여 두 개의 파일을 붙인 결과

그림 28은 그림 27과 달리 addition 파일의 내용 표현 방식이 원본의 페이지 설정을 따르는 것을 볼 수 있습니다. 이와 같이, 같은 구역에서 다음 페이지에 추가하기를 원하는 경우 해당 옵션을 사용합니다.

2. 필드에서 데이터 가져오기

1) 필드

필드는 한글 워드 프로세서에서 전체 문서 중 특정 위치나 내용을 표시하는 기능입니다. 필드는 주로 문서를 생성한 이후 내용을 변경하거나 가져오기 위해 사용합니다.



그림 4-17. 누름틀 사용 예제

한글 워드프로세서에는 위 그림과 같이 누름틀 필드 외에도 다양한 필드들이 있으며, 그리기 객체와 표 셀에 필드 이름을 설정하여 필드 기능을 적용하여 사용할 수 있습니다. 다음과 같은 방법으로 필드를 생성하거나 필드 이름을 입력할 수 있습니다.

그림 4-18. 한글 워드프로세서의 [필드 입력]



그림 4-19. 한글 워드프로세서의 [표/셀 속성]을 사용한 필드 이름 설정

위와 같은 방법으로 설정한 필드를 갖는 HWP 파일에서 필드 정보를 가져오기 위해서는 “HWPGenFieldGetter” 클래스에 포함된 다음의 함수를 사용합니다.

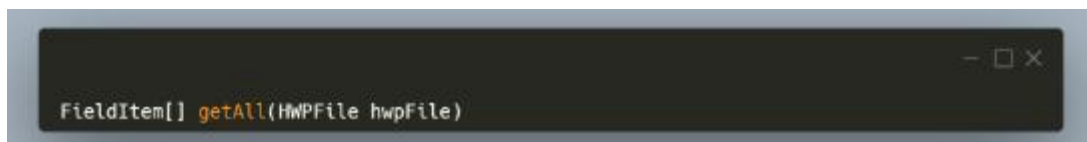


그림 4-20. HWP 파일 내 모든 필드 가져오기

위 함수를 사용하면 매개변수로 입력 받은 HWP 파일 내부에 있는 모든 필드들에 대한 정보를 반환합니다. 이 함수는 “FieldItem” 클래스의 배열 타입으로 반환합니다.

FieldItem 클래스는 다음과 같은 함수를 갖습니다.

함수	설명
FieldType type()	필드 타입(enum FieldType)을 반환한다. <i>주의) 사각형, 타원 등에 그리기 개체의 하위 타입은 생략하였으니 참고하시길 바랍니다.</i> <pre>public enum FieldType { ClickHere, // 누름틀</pre>

	Cell, // 셀에 설정된 필드 Gso, // 그리기개체에 설정된 필드 ETC. // 이외의 필드 }
String name()	필드 이름을 반환한다.
String text()	필드가 가지고 있는 텍스트를 반환한다.

표 4-1. FiledItem 클래스의 함수

예를 들어 "필드1"으로 설정한 누름틀 필드, "필드2"로 설정한 셀, "필드3"으로 설정한 셀, "필드4"로 설정한 글상자 개체를 갖는 "4_fields.hwp" 파일이 있다면,

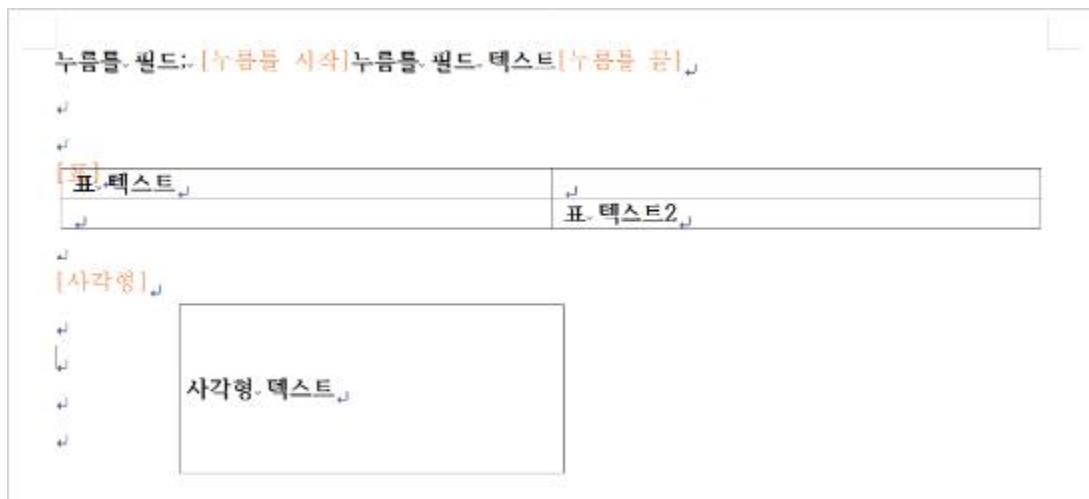


그림 4-21. 4_fields.hwp 문서

해당 파일을 읽고 필드 정보를 가져와 출력하는 아래의 JAVA 코드를 실행합니다.



그림 4-22. HWP 파일을 읽고 필드 정보를 가져와 출력하기

다음과 같은 결과를 획득할 수 있습니다.

ClickHere, 필드1, 누름틀 필드 텍스트

Cell, 필드2, 표 텍스트
 Cell, 필드3, 표 텍스트2
 Gso, 필드4, 사각형 텍스트

표 4-2. 그림 34 예제 실행 결과

getAll() 함수를 통해 획득한 FileItem[] 배열 내의 항목 순서는 HWP 파일 내부의 순서로 한글 워드프로세서나 뷰어 등에서 보이는 순서와 다를 수 있습니다.

2) 조건에 맞는 필드 가져오기

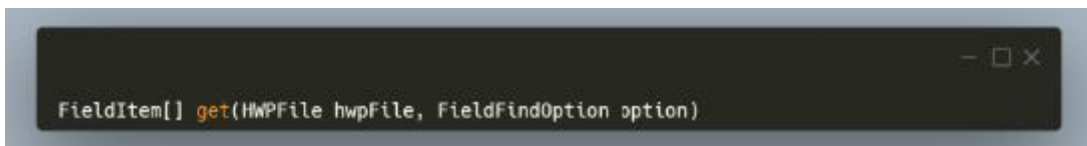


그림 4-23. 조건에 맞는 필드 가져오기 예제

이 함수는 FieldFindOption 클래스에 설정된 조건에 맞는 필드를 반환합니다.

FieldFindOption 클래스는 다음과 같은 함수를 갖습니다.

함수	설명
void nameToFind(String nameToFind)	찾을 이름을 설정한다. nameToField가 null이면 이름으로 찾지 않는다.
void onlyFirst(boolean onlyFirst)	첫번째 이름만을 찾을 것인지 여부를 설정한다.
void findField(boolean findField)	누름틀, 기타 필드를 찾을 것인지 여부를 설정한다.
void findGso(boolean findGso)	그리기 개체를 찾을 것인지 여부를 설정한다.
void findCell(boolean findCell)	표 셀을 찾을 것인지 여부를 설정한다.

표 4-3. FieldFindOption 클래스 제공 함수

예를 들어 4_fields.hwp 파일에서 필드 이름이 "필드2"를 찾는 JAVA 코드를 실행하면,

```

HWPFile hwpFile2 = HWPGenLoader.fromFilepath("4_fields.hwp");

FieldFindOption option = new FieldFindOption();
option.nameToFind("필드2");
option.onlyFirst(false);
option.findField(true);
option.findGso(true);
option.findCell(true);

FieldItem[] fieldItems = HWPGenFieldGetter.get(hwpFile2, option);

for (FieldItem fi : fieldItems) {
    System.out.println(fi.type() + ", " + fi.name() + ", " + fi.text() );
}

```

그림 4-24. 필드명이 "필드2"인 필드의 정보 가져와서 출력하기

아래와 같은 결과를 얻을 수 있습니다.

Cell, 필드2, 표 텍스트

표 4-4. 그림 36의 실행 결과

위와 같은 방식으로 한글 문서의 셀에 적용된 필드를 찾기 위해서는 아래와 같은 JAVA 코드를 작성하여,

```

HWPFile hwpFile = HWPGenLoader.fromFilepath("4_fields.hwp");

FieldFindOption option = new FieldFindOption();
option.nameToFind(null);
option.onlyFirst(false);
option.findField(false);
option.findGso(false);
option.findCell(true);

FieldItem[] fieldItems = HWPGenFieldGetter.get(hwpFile, option);

for (FieldItem fi : fieldItems) {
    System.out.println(fi.type() + ", " + fi.name() + ", " + fi.text() );
}

```

그림 4-25. 셀에 적용된 필드의 정보를 가져와서 출력하기

실행하면 다음과 같은 결과를 얻을 수 있습니다.

Cell, 필드2, 표 텍스트
Cell, 필드3, 표 텍스트2

표 4-5. 그림 37의 출력 결과

3. 필드에 정보 설정하기

HWP 파일이 포함하는 필드에 정보를 설정하기 위해서는 "HWPGenFieldSetter"클래스에 포함된 set() 함수를 사용합니다.

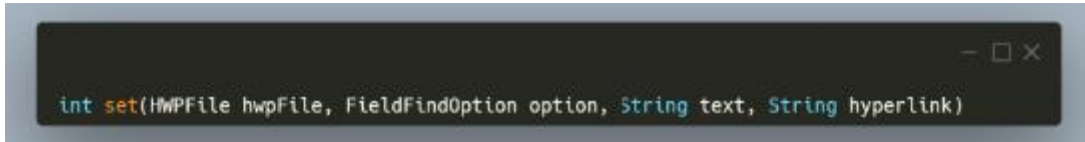


그림 4-26. set() 함수

set() 함수는 설정할 필드를 포함하는 HWP 파일 객체(hwpFile), 설정할 필드를 찾을 수 있는 조건 옵션(option), 필드에 설정할 텍스트(text) 그리고, 필드에 설정할 하이퍼링크(hyperlink)를 매개변수로 갖습니다.

예를 들어, "4_fields.hwp" 파일에 필드 이름이 "필드2"인 필드를 찾아 "새로운 필드 값"이라는 텍스트를 설정하고, "setfield_result.hwp"로 저장하기 위해서는 아래와 같은 JAVA 코드를 작성하여

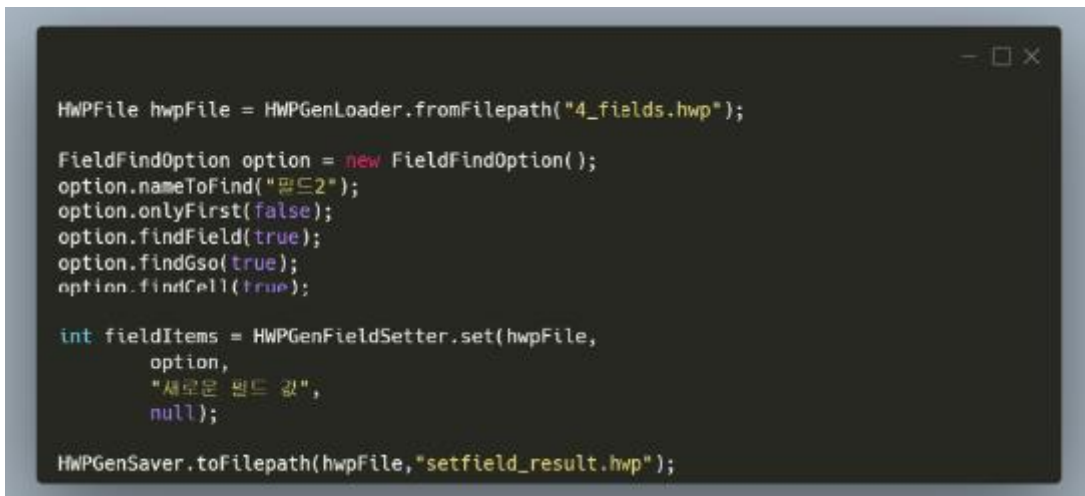


그림 4-27. 필드에 텍스트 설정하기

실행하면 다음과 같은 결과를 얻을 수 있습니다.

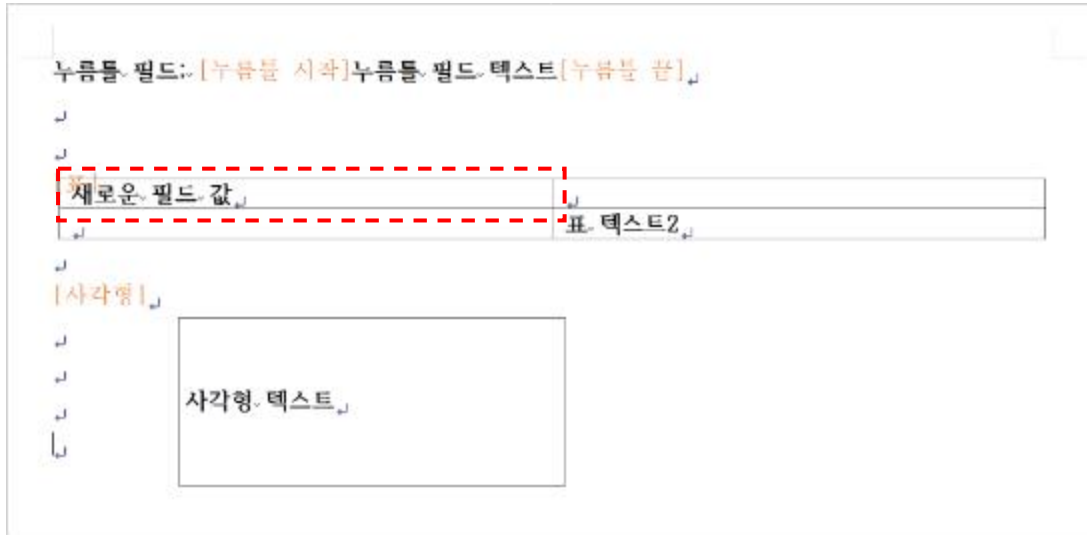


그림 4-28. 필드에 텍스트 설정 코드 실행 결과

set() 함수는 텍스트와 별도로 하이퍼링크를 매개변수로 넘겨줄 수 있습니다. 다음과 같이 텍스트와 함께 하이퍼링크를 매개변수로 넘겨주면,

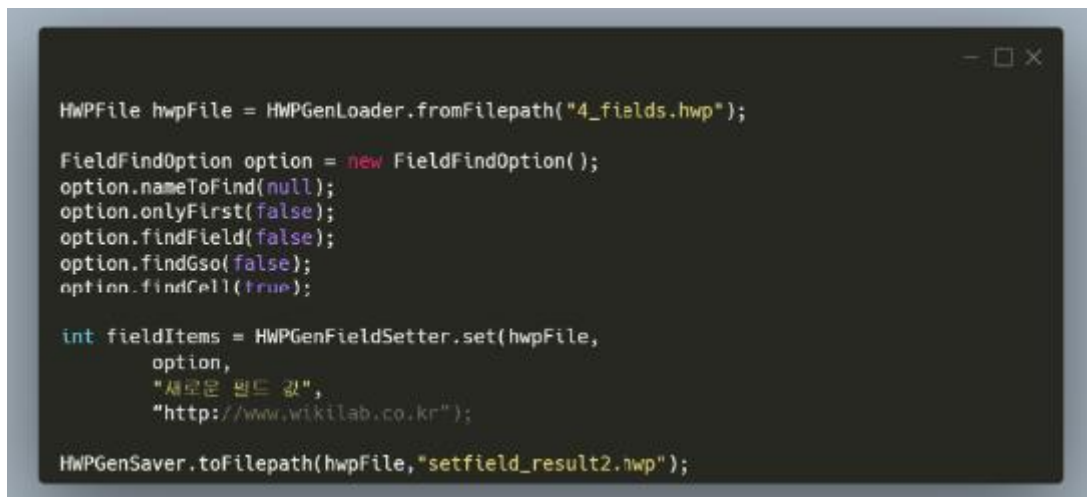


그림 4-29. 셀에 포함된 필드의 텍스트와 하이퍼링크 설정하기

아래와 같은 결과를 얻을 수 있습니다.



그림 4-30. 하이퍼링크 설정 코드 실행 결과

4. 표 정보 가져오기

1) 표 이름 설정하기

HWP 파일 내부 표의 정보를 가져오거나 설정하기 위해서는 우선 특정 표를 선택합니다. 한글 프로그램의 표는 필드를 선택할 수 있는 필드 이름을 설정하는 기능이 없습니다. 따라서, JAVA 프로그램 상에서 특정한 표를 선택하기 어렵습니다.

이 문제를 해결하기 위해 한글로젠은 표 이름이라는 개념을 사용합니다. 표 이름은 표의 첫 번째 셀 필드 이름을 표 이름으로 정의하여 특정 표를 찾는 방법입니다. 표의 첫 번째 셀을 선택한 후 [표/셀 속성]창에서 필드 이름을 설정하여 이름을 표 이름으로 정의합니다. 한글로젠은 이와 같이 정의된 표 이름을 바탕으로 HWP 파일 내부에서 표를 검색합니다.

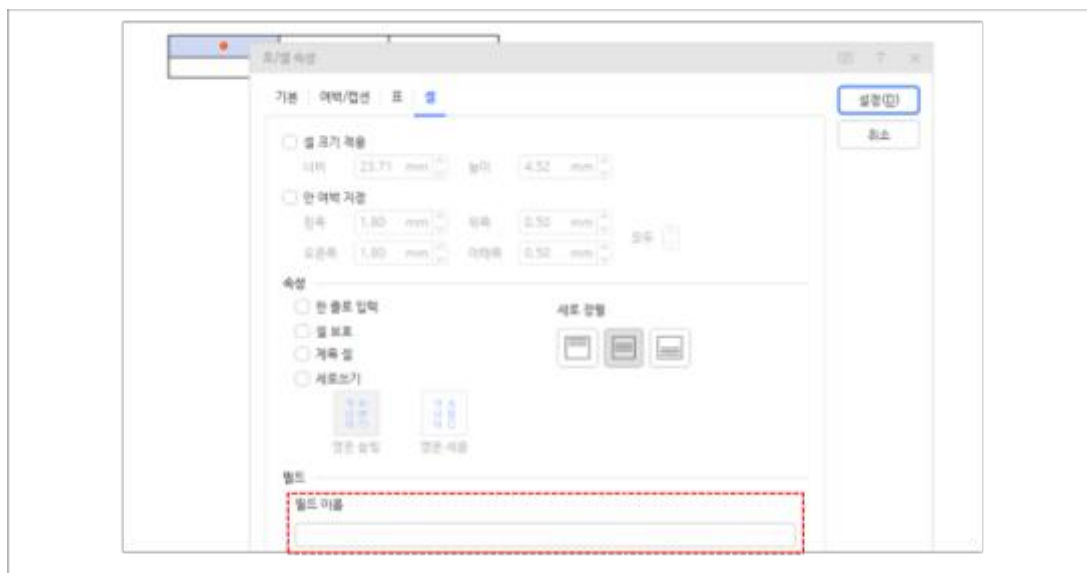


그림 4-31. 표/셀 속성의 필드 사용 예시

위와 같은 방법으로 정의된 표 이름은 한글로젠에서 다음과 같이 사용합니다.

```
String[][] tableText = HWPGenTableGetter.getAllCellText(hwpFile, "표이름1");
```

그림 4-32. 표의 텍스트 가져오기 예시

getAllCellText() 함수는 표의 모든 셀이 가지고 있는 텍스트를 String[][] 형태로 반환합니다. 이 함수가 반환하는 이차원 배열은 [행의 개수][열의 개수]의 원소를 갖습니다. 이 함수는 표 이름(tableName)으로 표를 찾지 못하면 null을 반환합니다.

```
String[][] getAllCellText(HWPFile hwpFile, String tableName)
```

그림 4-33. getAllCellText() 함수 사용 예시

표 1	A 필드 1		B 필드 2
C 필드 3	A 필드 1	D 필드 4	

그림 4-34. table.hwp 파일 내용

그림 46과 같은 표를 갖는 "table.hwp" 파일에서 표(테이블) 이름이 "표1"인 표를 찾아 전체 셀들의 텍스트를 획득하기 위해서는 아래의 JAVA 코드를 작성하여,



```

HWPFile hwpFile = HWPGenLoader.fromFilepath("table.hwp");

String[][] tableText = HWPGenTableGetter.getAllCellText(hwpFile2, "표1");

StringBuffer sb = new StringBuffer();
for (String[] rowText : tableText) {
    for (String cellText : rowText) {
        System.out.print("{ " + cellText + " } ");
    }
    System.out.print("\n");
}

```

그림 4-35. 이름이 "표1"인 표의 전체 텍스트 출력하기 예시

실행하면 다음과 같은 결과를 얻을 수 있습니다.

{	{A}	}	{	}
{	{null}	}	{	{B}
{C}	{null}	}	{D}	}

표 4-6. 그림 47의 실행 결과

table.hwp의 표는 3행 4열의 형태이기 때문에 getAllCellText() 함수는 String[3][4] 형태의 문자열 배열을 반환합니다. 배열의 [1][1], [2][1] 값은 null 임을 확인할 수 있는데, 이는 2열부터 4행 2열까지 셀이 병합되었기 때문입니다. 배열 원소의 null 값 여부를 판단하여 셀 병합 상태를 확인할 수 있습니다.

2) 표 전체의 필드 이름 가져오기



```

String[][] getAllCellFieldName(HWPFile hwpFile, String tableName)

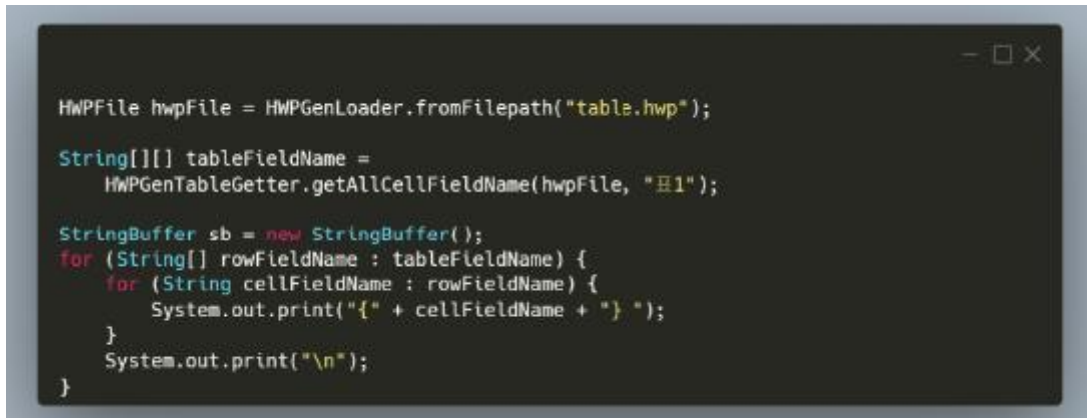
```

그림 4-36. getAllCellFieldName() 함수

getAllCellFieldName() 함수는 표 전체의 필드 이름을 String[][] 형태로 반환합니다. getAllCellText() 함수와 동일하게 [행 개수][열 개수] 개의 원소를 갖는 문자열 배열을 반환합니다. 이 함수는 표 이름(tableName)으로 표를 찾지 못하면 null을 반환합니다.

예를 들어 "table.hwp" 파일을 읽고 표 이름이 "표1"인 표를 찾아 전체 셀의 필드 이름을

가져오기 위해서는 아래와 같은 JAVA 코드를 작성하여,



```

HWPFile hwpFile = HWPGenLoader.fromFilepath("table.hwp");

String[][] tableFieldName =
    HWPGenTableGetter.getAllCellFieldName(hwpFile, "표1");

StringBuffer sb = new StringBuffer();
for (String[] rowFieldName : tableFieldName) {
    for (String cellFieldName : rowFieldName) {
        System.out.print("{ " + cellFieldName + " } ");
    }
    System.out.print("\n");
}
    
```

그림 4-37. 셀의 필드 이름 가져오기 예시

실행한 결과 다음과 같은 결과를 얻을 수 있습니다.

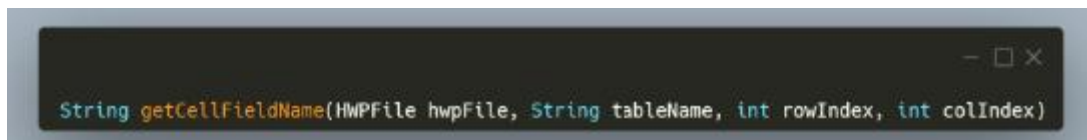


```

{표1} {필드1} {} {}
{} {null} {} {필드2}
{필드3} {null} {필드4} {}
    
```

표 4-7. 그림 49의 실행 결과

3) 셀의 필드 이름 가져오기



```

String getCellFieldName(HWPFile hwpFile, String tableName, int rowIndex, int colIndex)
    
```

그림 4-38. getCellFieldName() 함수

특정 셀의 필드 이름을 가져오기 위해서 getCellFieldName() 함수를 사용합니다. 이 함수는 셀의 필드 이름을 반환하는 것을 제외하면 getCellText() 함수와 유사하게 동작합니다.

예를 들어, "table.hwp" 파일을 읽어서 3행 1열 셀의 필드 이름을 가져오기 위해서는 다음과 같은 JAVA 코드를 작성하여,

```

HWPFile hwpFile = HWPGenLoader.fromFilepath("table.hwp");

String cellFieldName =
    HWPGenTableGetter.getCellFieldName(hwpFile2, "표1", 2, 0);

System.out.println(cellFieldName);

```

그림 4-39. 표 이름이 "표1"의 3행 1열 셀의 필드 이름 출력 예시

실행 결과는 아래와 같이 필드명 "필드3"을 얻을 수 있습니다.

필드3

표 4-8. 그림 51의 실행 결과

5. 표에 정보 설정하기

1) CSV 데이터

CSV는 Comma Separated Value의 약자로, 쉼표를 기준으로 항목을 구분하여 저장한 데이터를 말합니다.

김OO,89,65,78,이OO,77,77,77,박OO,77,77,77,최OO,60,62,63

표 4-9. 쉼표로 구분되는 CSV 데이터 예시

이와 같이 쉼표만으로 데이터를 구분하면 데이터 자체가 쉼표를 포함하는 경우 문제가 발생합니다. 이 문제를 해결하기 위해 쉼표 이외에 특정한 텍스트(TEXT)로 분리하게 되었습니다. CSV라는 용어는 쉼표를 포함한 특정 텍스트로 분리된 값을 통칭하는 의미로 사용하게 되었습니다.

김OO[!]89[!]65[!]78[!]이OO[!]77[!]77[!]77[!]박OO[!]77[!]77[!]77[!]최OO[!]60[!]62[!]63

표 4-10. 특정 텍스트로 구분되는 CSV 데이터 예시

[!]로 분리된 데이터를 사용하여 아래와 같은 결과를 얻기 위해서는 데이터의 값을 분리하는 것과 별도로, 표의 행 단위로 분리해야 합니다.

김OO[!]89[!]65[!]78[@]이OO[!]77[!]77[!]77[@]박OO[!]77[!]77[!]77[@]최OO[!]60[!]62[!]63

표 4-11. 필드 구분자와 레코드 구분자가 적용된 CSV 데이터 예시

이를 위해 한글로젠에서는 “필드 구분자”, “레코드 구분자” 개념을 사용합니다. 필드 구분은 구분자 [!]를 사용하여 데이터를 분리하고, 레코드 구분은 구분자 [@]를 사용하여 데이터를 행 단위로 분리하여 묶어주는 역할을 합니다.

한글로젠은 이와 같이 필드 구분자와 레코드 구분자로 분리된 CSV 데이터를 사용하여 HWP 파일 내부 표에 데이터를 삽입할 수 있습니다.

2) CSV 데이터 삽입하기

HWP 파일 안에 있는 표(Table)에 CSV 데이터 삽입을 위해 “HWPGenTableSetter”클래스의 insertCSVData() 함수를 사용합니다.

```
void insertCSVData(HWPFile hwpFile, String tableName,
    CSVDataSettingInfo settingInfo, String csvData) throws Exception
```

그림 4-40. insertCSVData() 함수 사용 예시

위 함수는 HWP 파일 객체(hwpFile), 표 이름(tableName), CSV 데이터설정 정보(settingInfo), CSV 데이터(csvData)를 매개변수로 사용합니다. HWP 파일 객체(hwpFile) 매개변수는 설정할 표를 포함하는 HWP 파일을 가리키며, 표 이름(tableName)은 HWP 파일 내에서 설정할 표를 찾을 수 있는 이름입니다. 이 매개변수들은 표와 관련된 모든 함수에서 동일하게 처리됩니다. 이 부분에 대해서는 “표 이름 설정하기” 부분을 참고하시기 바랍니다.

이 함수의 이해를 돕기 위해 다음과 같은 표 이름이 “성적테이블”인 표를 갖는 “원본.hwp” 파일을 예제로 사용합니다.

학생 성적 리스트			
이름	국어	영어	수학

그림 4-41. 이름이 “성적테이블”인 표를 갖는 “원본.hwp”

CSV 데이터 설정 정보(settingInfo)는 CSV 데이터(csvData)를 표에 설정하기 위한 정보를 가지고 있습니다. CSV 데이터 설정 정보는 CSVDataSettingInfo 클래스로 표현되고, 이 클래스는 다음과 같은 함수를 가지고 있습니다.

함수	설명
void repeatingRow(int startIndex, int endIndex)	표에서 데이터가 반복되는 행 구간을 설정한다.
void dataSeparator(String fieldSeparator, String recordSeparator)	필드 구분자와 레코드 구분자를 설정한다.
void addCellSetting(int dataIndex, int rowIndex, int colIndex)	셀에 데이터를 설정한다.

표 4-12. CSVDataSettingInfo 클래스의 함수 목록

repeatingRow() 함수는 HWP 파일의 표에서 데이터가 반복될 구간을 설정합니다.

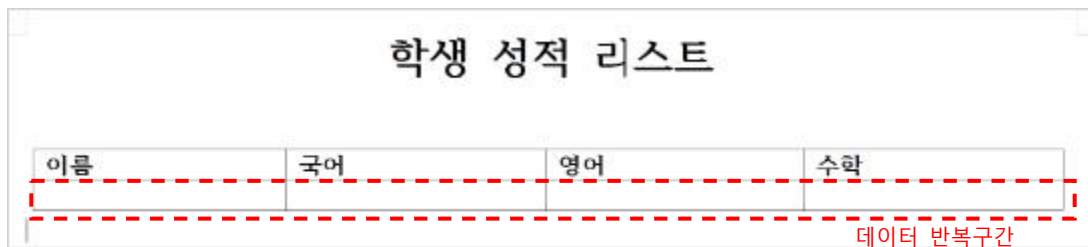


그림 4-42. 표에서 데이터 반복 행 구간

“원본.hwp”파일에서 데이터 반복 구간은 그림 54와 같이 두 번째행입니다. repeatingRow() 함수에서 startIndex를 1(0부터 시작), endIndex를 1(0부터 시작)로 설정합니다.

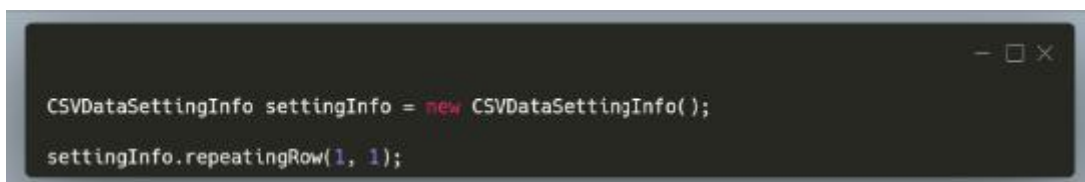


그림 4-43. 데이터 반복 구간을 두 번째 행으로 설정하기

dataSeparator() 함수는 이전 절에서 설명한 필드 구분자와 레코드 구분자를 설정하는 함수입니다. 필드 구분자를 [!], 레코드 구분자를 [@]로 설정하려면 다음과 같이 JAVA 코드를 작성합니다.



```

CSVDataSettingInfo settingInfo = new CSVDataSettingInfo();
settingInfo.dataSeparator("[!]", "[@]");

```

그림 4-44. 필드 구분자 [!], 레코드 구분자 [@] 설정

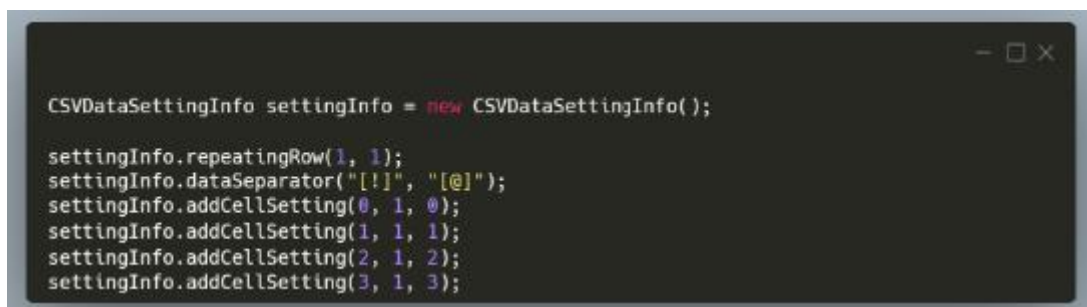
addCellSetting() 함수는 셀에 표현하는 데이터를 설정합니다. 이 함수는 데이터 인덱스(dataIndex), 행 인덱스(rowIndex), 열 인덱스(colIndex)를 매개변수로 갖고, 행 인덱스(rowIndex)와 열 인덱스(colIndex)에 해당하는 셀에 데이터 인덱스(dataIndex) 번째 데이터를 설정합니다. 데이터 인덱스(dataIndex)는 CSV 데이터에서 데이터가 표현된 순서를 나타냅니다.

표 4-11의 CSV 데이터는 아래와 같은 데이터 인덱스를 갖습니다.

데이터 인덱스	데이터			
0	김OO	이OO	박OO	최OO
1	89	77	88	60
2	65	77	88	62
3	78	77	88	63

표 4-13. 데이터 인덱스

[그림 4-41]의 표에 데이터를 설정하기 위해 다음과 같은 JAVA 소스 코드를 작성할 수 있습니다.



```

CSVDataSettingInfo settingInfo = new CSVDataSettingInfo();

settingInfo.repeatingRow(1, 1);
settingInfo.dataSeparator("[!]", "[@]");
settingInfo.addCellSetting(0, 1, 0);
settingInfo.addCellSetting(1, 1, 1);
settingInfo.addCellSetting(2, 1, 2);
settingInfo.addCellSetting(3, 1, 3);

```

그림 4-45. 셀에 데이터 설정하기

전체 JAVA 소스 코드를 다음과 같이 작성하여,

```

HWPFile hwpFile = HWPGenLoader.fromFilepath("원본.hwp");

CSVDataSettingInfo settingInfo = new CSVDataSettingInfo();
settingInfo.repeatingRow(1, 1);
settingInfo.dataSeparator("[!]", "[@]");
settingInfo.addCellSetting(0, 1, 0);
settingInfo.addCellSetting(2, 1, 2);

String data = "김00[!]89[!]65[!]78[!]이00[!]70[!]71[!]72[!]박00[!]83[!]84[!]85[!]최  
00[!]66[!]67[!]68";

HWPGenTableSetter.insertCSVData(hwpFile, "성적테이블", settingInfo, data);

HWPGenSaver.toFilepath(hwpFile, "결과.hwp");

```

그림 4-46. CSV 데이터 삽입하기 전체 JAVA 소스 코드

실행하면 아래와 같은 결과를 얻을 수 있습니다.

이름	국어	영어	수학
김00	89	65	78
이00	77	77	77
박00	83	88	88
최00	67	67	68

그림 4-47. CSV 데이터 삽입 소스 코드 실행 결과

3) JSON 데이터 삽입하기

HWP 파일 안에 있는 표에 JSON 데이터를 삽입하기 위해서는 insertJSONData() 함수를 사용하여야 합니다.

```

void insertJSONData(HWPFile hwpFile, String tableName,
    JSONDataSettingInfo settingInfo, String jsonData) throws Exception

```

그림 4-48. insertJSONData() 함수 사용 예시

본 함수는 HWP 파일 객체(hwpFile), 표 이름(tableName), JSON 데이터 설정 정보(settingInfo), JSON 데이터(jsonData)를 매개변수로 갖습니다. HWP 파일 객체와 표 이름 매개변수는 표와 관련된 모든 함수들과 동일하게 처리합니다. 상세 내용은 "표 이름 설정하기" 절을 참고하시길 바랍니다.

JSON 데이터 설정 정보는 JSON 데이터를 표에 설정하기 위한 정보를 가지고 있습니다. JSON 데이터 설정 정보는 JSONDataSettingInfo 클래스로 표현되고, 이 클래스는 다음과 같은 함수를 포함하고 있습니다.

함수	설명
void repeatingRow(int startIndex, int endIndex)	표에서 데이터가 반복되는 행 구간을 설정한다.
void rootPath(String rootPath)	JSON 루트 경로를 설정한다.
void addCellSetting(String dataPath, int rowIndex, int colIndex)	셀에 데이터를 설정한다.

표 4-14. JSONDataSettingInfo 클래스 제공 함수

repeatingRow() 함수는 HWP 파일의 표에서 데이터가 반복될 구간을 설정합니다. 이 함수는 CSVDataSettingInfo 클래스와 동일합니다. 자세한 내용은 CSVDataSettingInfo 클래스를 설정하는 부분을 참고하시길 바랍니다.

rootPath() 함수는 insertJSONData()에서 받은 전체 JSON 데이터 중 HWP 파일에 삽입할 부분을 설정합니다. JSON 데이터는 무한 확장이 가능한 노드를 갖는 계층적 구조입니다. 이러한 JSON 데이터 노드 중 데이터베이스의 레코드와 같이 동일한 의미/형태가 반복되는 부분의 노드를 루트 노드로 선택하면 반복되는 데이터로써 사용할 수 있습니다.

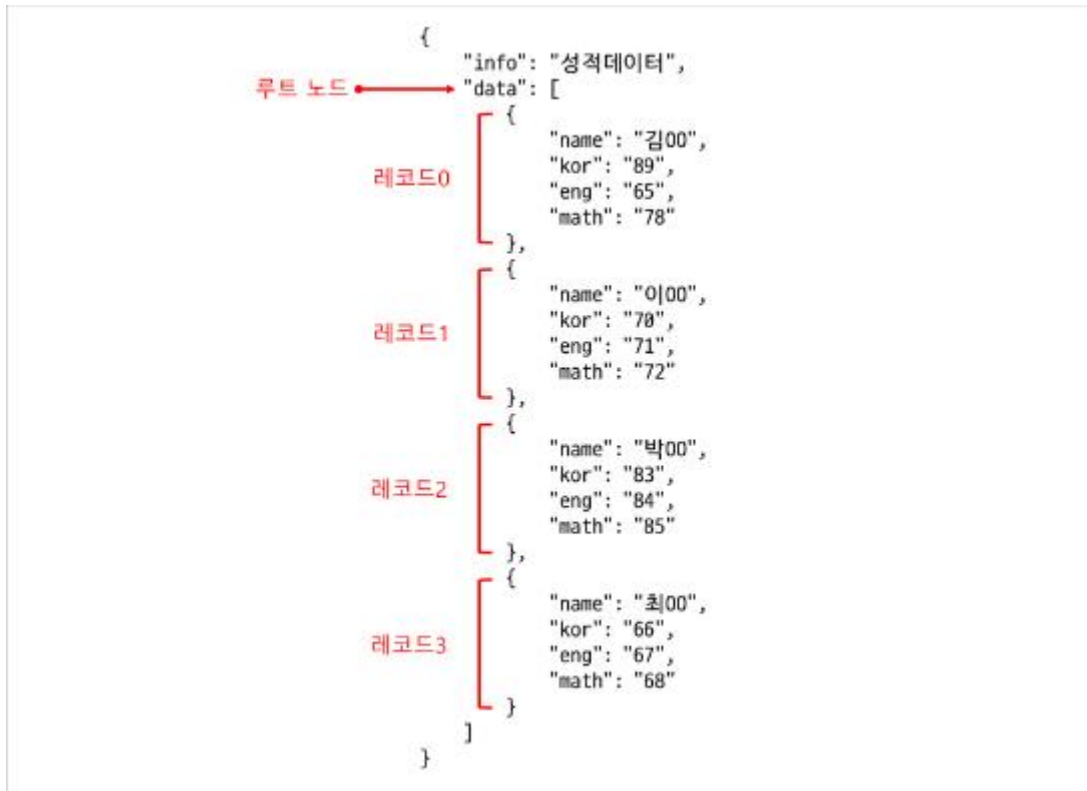


그림 4-49. JSON 데이터

위와 같은 구조의 JSON 데이터는 "data" 노드를 루트 노드로 선택할 수 있습니다. 루트 노드는 JSON 배열 객체를 값으로 가져야 하고, 이 배열은 반복되는 레코드 데이터를 원소로 구성됩니다. [그림 61]에서 "data" 노드를 루트로 설정하면 4개의 레코드를 갖는 데이터를 표에 삽입할 수 있습니다.

루트 노드로 선택한 "data" 노드의 경로를 표현하기 위해, 한글로젠은 오픈소스로 개발된 "Jayway JsonPath" 라이브러리를 사용합니다. 이 라이브러리에서 제공하는 JSON-Path는 XML의 X-Path와 유사하게 동작합니다. 이 라이브러리에 대해서는 오픈소스 홈페이지 "<https://github.com/json-path/JsonPath>"를 참고하시기 바랍니다.

JSON 데이터의 루트 경로를 설정하기 위해 다음과 같은 JAVA 소스 코드를 작성합니다.

```

JSONDataSettingInfo settingInfo = new JSONDataSettingInfo();
settingInfo.rootPath("$.data");

```

그림 4-50. JSON 데이터의 루트 경로 설정 예시

addCellSetting() 함수는 셀에 표현하는 데이터를 설정합니다. 이 함수는 데이터 경로(dataPath), 행 인덱스(rowIndex), 열 인덱스(colIndex)를 매개변수로 하며, 행 인덱스(rowIndex)와 열 인덱스(colIndex)에 해당하는 셀의 데이터 경로에 있는 데이터를 설정합니다. 데이터 경로는 루트 경로와 동일하게 "Jayway JsonPath" 라이브러리의 JSON-Path 형식으로 표현됩니다.

이 경로는 JSON 데이터의 전체 루트로부터 경로가 아닌, rootPath() 함수로 설정한 루트 경로로부터의 경로로 입력합니다. 아래와 같이 이름 데이터의 경로는 "\$.data[0].name"(JSON 데이터의 전체 루트로부터의 경로)가 아닌 "\$.name"(rootPath() 함수로 설정한 루트 경로로부터의 경로)으로 설정합니다.

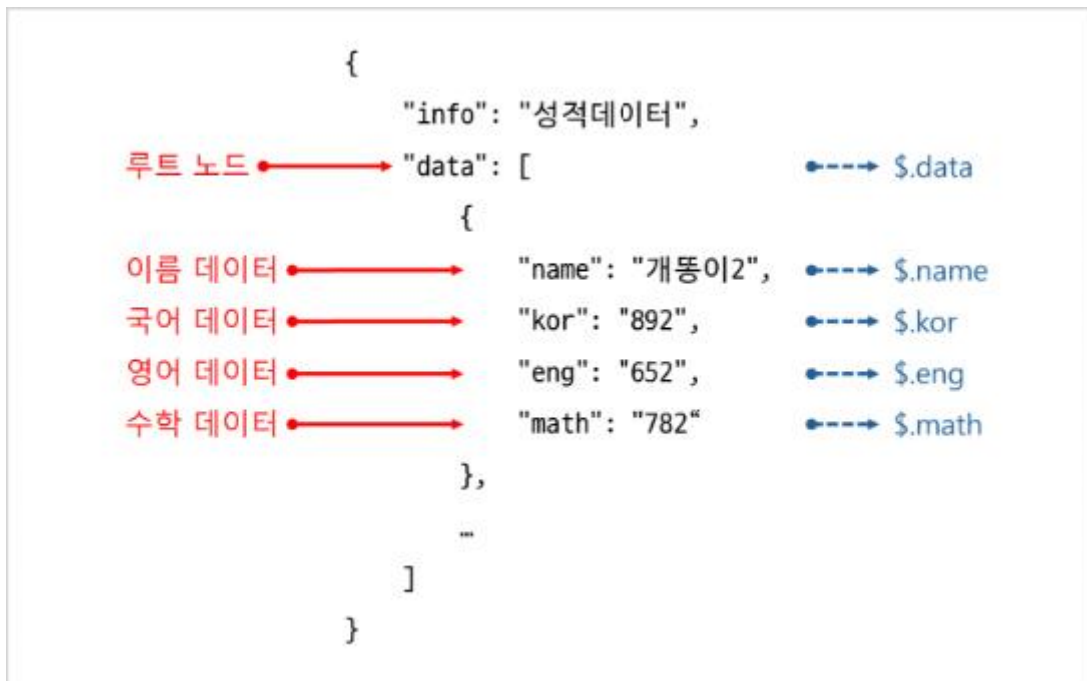


그림 4-51. JSON 데이터 예시 및 구조

위 데이터를 표에 설정하기 위해서는 다음과 같은 JAVA 소스 코드를 작성합니다.

```
JSONDataSettingInfo settingInfo = new JSONDataSettingInfo();

settingInfo.repeatingRow(1, 1);
settingInfo.rootPath("$.data");
settingInfo.addCellSetting("$.name", 1, 0);
settingInfo.addCellSetting("$.kor", 1, 1);
settingInfo.addCellSetting("$.eng", 1, 2);
settingInfo.addCellSetting("$.math", 1, 3);
```

그림 4-52. 표 셀에 데이터 설정하기 예시

전체 JAVA 소스 코드를 다음과 같이 작성하여,

```
HWPFile hwpFile = HWPGenLoader.fromFilepath("원본.hwp");

JSONDataSettingInfo settingInfo = new JSONDataSettingInfo();
settingInfo.repeatingRow(1, 1);
settingInfo.rootPath("$.data");
settingInfo.addCellSetting("$.name", 1, 0);
settingInfo.addCellSetting("$.kor", 1, 1);
settingInfo.addCellSetting("$.eng", 1, 2);
settingInfo.addCellSetting("$.math", 1, 3);

String data = "{ \"info\": \"성적데이터\", \"data\": [{ \"name\": \"김00\", \"kor\": \"89\", \"eng\": \"65\", \"math\": \"78\" }, { \"name\": \"이00\", \"kor\": \"70\", \"eng\": \"71\", \"math\": \"72\" }, { \"name\": \"박00\", \"kor\": \"83\", \"eng\": \"84\", \"math\": \"85\" }, { \"name\": \"최00\", \"kor\": \"66\", \"eng\": \"67\", \"math\": \"68\" } ] }";

HWPGenTableSetter.insertJSONData(hwpFile, "성적테이블", settingInfo, data);

HWPGenSaver.toFilepath(hwpFile, "결과.hwp");
```

그림 4-53. JSON 데이터 삽입하기 예시

실행하면 다음과 같은 결과를 얻을 수 있습니다.

학생 성적 리스트			
이름	국어	영어	수학
김00	89	65	78
이00	77	77	77
박00	83	84	85
최00	66	67	68

그림 4-54. JSON 데이터 삽입하기 소스 코드 실행 결과

4) 셀 설정하기

표 안의 셀 설정을 위해서 setCellText() 함수를 사용합니다.

```
boolean setCell(HWPFile hwpFile, String tableName, int rowIndex, int colIndex,
    String text,
    String hyperlink,
    String fieldName) throws UnsupportedOperationException
```

그림 4-55. setCell() 함수 사용 예시

이 함수는 HWP 파일 객체(hwpFile), 표 이름(tableName), 행 인덱스(rowIndex), 열 인덱스(colIndex), 설정할 텍스트(text), 설정할 하이퍼링크(hyperlink)를 매개변수로 가집니다. HWP 파일 객체와 표 이름 매개변수는 표와 관련된 모든 함수에서 동일하게 처리됩니다. 이 부분에 대해서는 “표 이름 설정하기” 절을 참고하시기 바랍니다.

이 함수는 HWP 파일 객체와 표 이름 매개변수로 HWP 표 객체를 찾은 후, rowIndex번째 행, colIndex 번째 열에 속한 셀을 찾아 그 셀에 텍스트, 하이퍼링크, 필드 이름을 설정합니다.

예를 들어, [그림 53]의 표에 2행 1열 셀의 텍스트를 “한컴이노스트림”, 하이퍼링크를 “https://www.hancomins.com”, 필드 이름을 “회사명”으로 설정하기 위해서는 다음과 같은 JAVA 소스 코드를 작성합니다.

```
HWPFile hwpFile = HWPGenLoader.fromFilepath("원본.hwp");

.setCell(hwpFile, "성적테이블", 1, 0,
    "한컴이노스트림",
    "https://www.hancomins.com",
    "회사명");

HWPGenSaver.toFilepath(hwpFile, "결과.hwp");
```

그림 4-56. 셀 설정하기

JAVA 소스 코드를 실행하면 아래와 같이 2행 1열 셀의 텍스트를 “한컴이노스트림”, 하이퍼링크를 “https://www.hancomins.com”, 필드 이름을 “회사명”으로 설정된 것을 확인할 수 있습니다.

그림 4-57. [그림 4-56] 소스 코드 실행 결과

6. API 리스트

1) HWPGenLoader 클래스

■ fromFilepath() 함수

원형	public static HWPFile fromFilepath(String filepath) throws Exception		
설명	파일 경로로부터 HWP 파일을 로딩한다.		
매개 변수	타입	이름	설명
	String	Filepath	로딩할 HWP 파일의 경로
리턴값	HWPFile: 로딩한 HWP 파일 객체		
예외	Exception: 파일을 로딩하는 중에 발생하는 오류		

■ formURL() 함수

원형	public static HWPFile fromURL(String url) throws Exception		
설명	URL로부터 HWP 파일을 로딩한다.		
매개 변수	타입	이름	설명
	String	url	로딩할 HWP 파일의 url 경로
리턴값	HWPFile: 로딩한 HWP 파일 객체		
예외	Exception: 파일을 로딩하는 중에 발생하는 오류		

■ formInputStream() 함수

원형	public static HWPFile fromInputStream(InputStream is) throws Exception		
설명	InputStream를 통해 HWP 파일을 로딩한다.		
매개 변수	타입	이름	설명
	InputStream	Is	HWP 파일을 로딩할 InputStream
리턴값	HWPFile: 로딩한 HWP 파일 객체		
예외	Exception: 파일을 로딩하는 중에 발생하는 오류		

■ formBase64() 함수

원형	public static HWPFile fromBase64(String base64) throws Exception		
설명	Base64 형식의 문자열로부터 HWP 파일을 로딩한다.		
매개 변수	타입	이름	설명
	String	base64	HWP 파일을 포함하는 base64 문자열
리턴값	HWPFile: 로딩한 HWP 파일 객체		
예외	Exception: 파일을 로딩하는 중에 발생하는 오류		

2) HWPGenSaver 클래스

■ toFilepath() 함수

원형	public static void toFilepath(HWPFile hwpFile, String filepath) throws Exception		
설명	HWP 파일을 파일 경로에 저장한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	저장할 HWP 파일 객체
	String	Filepath	저장할 파일 경로
리턴값			
예외	Exception: 파일을 저장하는 중에 발생하는 오류		

■ toOutputStream() 함수

원형	public static void toOutputStream(HWPFile hwpFile, OutputStream os) throws Exception		
설명	HWP 파일을 OutputStream을 통해 저장한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	저장할 HWP 파일 객체
	OutputStream	os	저장할 OutputStream
리턴값			

예외	Exception: 파일을 저장하는 중에 발생하는 오류
----	--------------------------------

■ toBase64() 함수

원형	public static String toBase64(HWPFile hwpFile) throws Exception		
설명	HWP 파일을 base64 형식의 문자열로 저장한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	저장할 HWP 파일 객체
리턴값	String: HWP 파일을 포함하는 base64 형식의 문자열		
예외	Exception: 파일을 저장하는 중에 발생하는 오류		

3) HWPGenInsertter 클래스

■ appendToEndOfBody() 함수

원형	public static void appendToEndOfBody(HWPFile origin, HWPFile addition) throws Exception		
설명	원본(origin) HWP 파일의 본문 끝에 추가내용(addition) HWP 파일을 붙인다.		
매개 변수	타입	이름	설명
	HWPFile	origin	원본 HWP 파일 객체
	HWPFile	addition	추가내용 HWP 파일 객체
	AppendOption	option	붙이기 옵션 public enum AppendOption { FromLastSection, // 마지막 구역에 붙임 FromNewSection, // 새로운 구역에 붙임 FromNewPage // 새로운 페이지에 붙임 }
리턴값			
예외	Exception: 파일의 내용을 복사하여 붙이는 중에 발생하는 오류		

4) HWPGenFieldGetter 클래스

■ getAll() 함수

원형	public static FieldItem[] getAll(HWPFile hwpFile)		
설명	HWP 파일 안에 포함된 모든 필드의 정보를 가져온다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	필드 정보를 가져올 HWP 파일 객체
리턴값	FieldItem[]: HWP 파일 안에 포함된 모든 필드 정보 HWP 파일 안에 필드가 없으면 FieldItem[0]를 리턴한다.		
예외			

■ get() 함수

원형	public static FieldItem[] get(HWPFile hwpFile, FieldFindOption option)		
설명	HWP 파일 안에 포함된 필드 중 조건에 맞는 필드의 정보를 가져온다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	필드 정보를 가져올 HWP 파일 객체
	FieldFindOption	option	필드 검색 조건
리턴값	FieldItem[]: HWP 파일 안에 포함된 필드 중 조건의 맞는 필드의 정보 HWP 파일 안에 조건에 맞는 필드가 없으면 FieldItem[0]를 리턴한다.		
예외			

5) FieldItem 클래스

- HWPGenFieldGetter 클래스의 함수에서 필드 정보를 반환한다.

■ type() 함수

원형	public FieldType type()
설명	필드 타입을 리턴한다.
매개 변수	
리턴값	FieldType : 필드 타입 <pre> public enum FieldType { ClickHere, // 누름틀 필드 Cell, // 셀에 설정된 필드 Gso, // 사각형 등 그리기 개체에 설정된 필드 ETC // 기타 필드 } </pre>
예외	

■ name() 함수

원형	public String name()
설명	필드 이름을 리턴한다.
매개 변수	
리턴값	String: 필드 이름
예외	

■ text() 함수

원형	public String text()
설명	필드 안에 포함된 텍스트를 리턴한다.
매개 변수	
리턴값	String: 필드 안에 포함된 텍스트
예외	

6) FieldFindOption 클래스

- 필드를 검색하는 조건을 설정합니다.

■ 생성자1

원형	public FieldFindOption()
설명	FieldFindOption 객체를 생성한다.
매개 변수	
리턴값	생성된 FieldFindOption 객체
예외	

■ 생성자2

원형	public FieldFindOption(String nameToFind, boolean onlyFirst, boolean findField, boolean findCell, boolean findGso)		
설명	FieldFindOption 객체를 생성한다.		
매개 변수	타입	이름	설명
	String	nameToFind	찾을 필드 이름, null이면 이름으로 찾지 않는다.
	boolean	onlyFirst	첫번째 것만 찾을 건지 여부. True이면 조건에 맞는 첫번째 필드를 찾을 후 검색을 종료한다.
	boolean	findField	누름틀 필드나 기타 필드들을 검색할 것인지 여부
	boolean	findCell	셀에 설정된 필드들을 검색할 것인지 여부
	boolean	findGso	사각형 등 그리기 개체에 설정된 필드들을 검색할 것인지 여부
리턴값	생성된 FieldFindOption 객체		
예외			

■ nameToFind() 함수

원형	public void nameToFind(String nameToFind)
설명	찾을 필드 이름을 설정한다.

매개 변수	타입	이름	설명
	String	nameToFind	찾을 필드 이름, null이면 이름으로 찾지 않는다.
리턴값			
예외			

■ onlyFirst() 함수

원형	public void onlyFirst(boolean onlyFirst)		
설명	첫번째 것만 찾을 건지 여부를 설정한다.		
매개 변수	타입	이름	설명
	boolean	onlyFirst	첫번째 것만 찾을 건지 여부. True이면 조건에 맞는 첫번째 필드를 찾을 후 검색을 종료한다.
리턴값			
예외			

■ findField() 함수

원형	public void findField(boolean findField)		
설명	누름틀 필드나 기타 필드들을 검색할 것인지 여부를 설정한다.		
매개 변수	타입	이름	설명
	boolean	findField	누름틀 필드나 기타 필드들을 검색할 것인지 여부
리턴값			
예외			

■ findCell() 함수

원형	public void findCell(boolean findCell)		
설명	셀에 설정된 필드들을 검색할 것인지 여부를 설정한다.		
매개 변수	타입	이름	설명
	boolean	findCell	셀에 설정된 필드들을 검색할 것인지 여부
리턴값			
예외			

■ findGso() 함수

원형	public void findGso(boolean findGso)		
설명	사각형 등 그리기 개체에 설정된 필드들을 검색할 것인지 여부를 설정한다.		

매개 변수	타입	이름	설명
	boolean	findGso	사각형 등 그리기 개체에 설정된 필드들을 검색할 것인지 여부
리턴값			
예외			

7) HWPGenFieldSetter 클래스

■ set() 함수

원형	public static int set(HWPFile hwpFile, FieldFindOption option, String text, String hyperLink) throws UnsupportedOperationException		
설명	HWP 파일 안에 조건에 맞는 필드의 정보를 설정한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	필드 정보를 설정할 HWP 파일 객체
	FieldFindOption	option	필드 검색 조건
	String	text	설정될 텍스트, null이면 텍스트와 하이퍼링크가 설정되지 않는다.
매개 변수	String	hyperLink	설정될 하이퍼링크, null이면 텍스트만 설정된다.
리턴값	int: 설정된 필드의 개수, 조건에 맞는 필드를 찾지 못하면 0을 리턴한다.		
예외	UnsupportedEncodingException: 매개변수 text, hyperLink을 UTF_16LE로 변경하는 중에 발생하는 오류		

8) HWPGenFieldSetter 클래스

■ getAllCellText() 함수

원형	public static String[][] getAllCellText(HWPFile hwpFile, String tableName)		
설명	표의 모든 셀의 텍스트를 리턴한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	표 정보를 가져올 HWP 파일 객체
매개 변수	String	tableName	표 이름
리턴값	String[][]: 표 안에 포함된 모든 셀의 텍스트를 [행 개수][열 개수] 크기의 2차원 문자열 배열로 리턴한다. 각 셀의 텍스트는 배열의 [행 인덱스][열 인덱스] 번째 원소로 저장되며, 병합되어 사라진 셀들의 배열 원소는 null 값을 가진다. 표 이름(tableName)에 해당하는 표를 찾지 못하면 null을 리턴한다.		
예외			

■ getAllCellFieldName() 함수

원형	public static String[][] getAllCellFieldName(HWPFile hwpFile, String tableName)		
설명	표의 모든 셀의 필드 이름을 리턴한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	표 정보를 가져올 HWP 파일 객체
	String	tableName	테이블 이름
리턴값	String[][]: 표 안에 포함된 모든 셀의 필드 이름을 [행 개수][열 개수] 크기의 2차원 문자열 배열, 각 셀의 필드 이름은 배열의 [행 인덱스][열 인덱스] 번째 원소로 저장되며, 병합되어 사라진 셀들의 배열 원소는 null 값을 가진다. 표 이름(tableName)에 해당하는 표를 찾지 못하면 null을 리턴한다.		
예외			

■ getCellText() 함수

원형	public static String getCellText(HWPFile hwpFile, String tableName, int rowIndex, int colIndex)		
설명	셀의 텍스트를 리턴한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	표 정보를 가져올 HWP 파일 객체
	String	tableName	표 이름
	Int	rowIndex	행 인덱스(0부터 시작됨)
	Int	colIndex	열 인덱스(0부터 시작됨)
리턴값	String: 셀의 텍스트. 표 이름(tableName)에 해당하는 표를 찾지 못하거나 행 인덱스(rowIndex)와 열 인덱스(colIndex)에 해당하는 셀을 찾지 못하면 null을 리턴한다.		
예외			

■ getCellFieldName() 함수

원형	public static String getCellFieldName(HWPFile hwpFile, String tableName, int rowIndex, int colIndex)		
설명	셀의 필드 이름을 리턴한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	표 정보를 가져올 HWP 파일 객체
	String	tableName	표 이름
	Int	rowIndex	행 인덱스(0부터 시작됨)
	Int	colIndex	열 인덱스(0부터 시작됨)
리턴값	String: 셀의 필드 이름, 표 이름(tableName)에 해당하는 표를 찾지 못하거나 행 인덱스(rowIndex)와 열 인덱스(colIndex)에 해당하는 셀을 찾지 못하면 null을 리턴한다.		

예외

9) HWPGenTableSetter 클래스

■ insertCSVData() 함수

원형	public static void insertCSVData(HWPFile hwpFile, String tableName, CSVDataSettingInfo settingInfo, String csvData) throws Exception		
설명	표에 CSV 데이터를 삽입한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	표 정보를 가져올 HWP 파일 객체
	String	tableName	표 이름
	CSVDataSettingInfo	settingInfo	CSV 데이터 설정 정보
	String	csvData	CSV 데이터
리턴값			
예외	Exception: 표를 찾지 못하였거나, CSV 데이터를 파싱 에러, 셀에 데이터 설정 오류 등		

■ insertJSONData() 함수

원형	public static void insertJSONData(HWPFile hwpFile, String tableName, JSONDataSettingInfo settingInfo, String jsonData) throws Exception		
설명	표에 JSON 데이터를 삽입한다.		
매개 변수	타입	이름	설명
	HWPFile	hwpFile	표 정보를 가져올 HWP 파일 객체
	String	tableName	표 이름
	JSONDataSettingInfo	settingInfo	JSON 데이터 설정 정보
	String	jsonData	JSON 데이터
리턴값			
예외	Exception: 표를 찾지 못하였거나, JSON 데이터를 파싱 에러, 셀에 데이터 설정 오류 등		

■ setCell() 함수

원형	public static boolean setCell(HWPFile hwpFile, String tableName, int rowIndex, int colIndex, String text, String hyperLink, String fieldName) throws UnsupportedOperationException		
설명	셀에 텍스트, 하이퍼링크, 필드 이름을 설정한다.		

매개 변수	타입	이름	설명
	HWPFile	hwpFile	표 정보를 가져올 HWP 파일 객체
	String	tableName	표 이름
	Int	rowIndex	행 인덱스(0부터 시작됨)
	Int	colIndex	열 인덱스(0부터 시작됨)
	String	Text	설정될 텍스트, null이면 텍스트와 하이퍼링크가 설정되지 않는다.
	String	hyperLink	설정될 하이퍼링크, null이면 텍스트만 설정된다.
리턴값	boolean: 설정 여부, 표 이름(tableName)에 해당하는 표를 찾지 못하거나 행 인덱스(rowIndex), 열 인덱스(colIndex)에 해당하는 셀을 찾지 못하면 False를 리턴한다.		
	예외		
	UnsupportedEncodingException: 매개변수 text, hyperLink을 UTF_16LE로 변경하는 중에 발생하는 오류		

10) HWPGenTableSetter 클래스

■ 생성자

원형	public CSVDataSettingInfo()
설명	CSVDataSettingInfo 객체를 생성한다.
매개 변수	
리턴값	생성된 CSVDataSettingInfo 객체
예외	

■ repeatingRow() 함수

원형	public void repeatingRow(int startIndex, int endIndex)		
설명	표의 반복 행을 설정한다.		
매개 변수	타입	이름	설명
	Int	startIndex	반복 시작 행의 인덱스(0부터 시작됨)
	Int	endIndex	반복 끝 행의 인덱스(0부터 시작됨)
리턴값			
예외			

■ dataSeparator() 함수

원형	public void dataSeparator(String fieldSeparator, String recordSeparator)
----	--

설명	CSV 데이터의 데이터 구분자를 설정한다.		
매개 변수	타입	이름	설명
	String	fieldSeparator	CSV 데이터의 필드 구분자
	String	recordSeparator	CSV 데이터의 레코드 구분자
리턴값			
예외			

■ addCellSetting() 함수

원형	public void addCellSetting(int dataIndex, int rowIndex, int colIndex)		
설명	셀 설정 정보를 추가한다.		
매개 변수	타입	이름	설명
	Int	dataIndex	셀에 설정할 CSV 데이터의 인덱스(0부터 시작됨)
	Int	rowIndex	설정할 셀의 행 인덱스(0부터 시작됨)
	Int	colIndex	설정할 셀의 열 인덱스(0부터 시작됨)
리턴값			
예외			

11) JSONDataSettingInfo 클래스

- 표에 JSON 데이터를 삽입하기 위한 설정 정보를 입력합니다.

■ 생성자

원형	public JSONDataSettingInfo()		
설명	JSONDataSettingInfo 객체를 생성한다.		
매개 변수			
리턴값	생성된 JSONDataSettingInfo 객체		
예외			

■ repeatingRow() 함수

원형	public void repeatingRow(int startIndex, int endIndex)		
설명	표의 반복 행을 설정한다.		
매개 변수	타입	이름	설명
	Int	startIndex	반복 시작 행의 인덱스(0부터 시작됨)
	Int	endIndex	반복 끝 행의 인덱스(0부터 시작됨)

리턴값	
예외	

■ rootPath() 함수

원형	public void rootPath(String rootPath)		
설명	JSON 데이터의 루트 노드 경로를 설정한다.		
매개 변수	타입	이름	설명
	String	rootPath	JSON 데이터의 루트 노드 경로
리턴값			
예외			

■ addCellSetting() 함수

원형	public void addCellSetting(String dataPath, int rowIndex, int colIndex)		
설명	셀 설정 정보를 추가한다.		
매개 변수	타입	이름	설명
	String	dataPath	셀에 설정할 JSON 노드의 경로
	Int	rowIndex	설정할 셀의 행 인덱스(0부터 시작됨)
	Int	colIndex	설정할 셀의 열 인덱스(0부터 시작됨)
리턴값			
예외			



V. 문제 해결 및 지원



V. 문제 해결 및 지원

1. 자주 묻는 질문(FAQ)

Q1) HWP 파일을 읽을 때 메모리 관련 오류가 발생합니다.

Answer) 해당 오류는 메모리 부족으로 발생하는 오류입니다.

JAVA에서는 JRE(Java Runtime Environment)에 옵션으로 메모리를 설정하여 사용 가능한 메모리 용량을 조절할 수 있습니다. 메모리가 부족하여 문제가 발생하며, 이 옵션을 설정하거나 하드웨어적으로 메모리를 증설하는 드로이 JAVA 시스템 운영에 대한 일반적 대응으로 해결할 수 있습니다.

Q2) HWPGenFieldGetter.getAll() 함수의 결과가 화면에 표현된 순서와 맞지 않습니다.

Answer) 한글 프로그램 등에서 그려지는 순서는 HWP 파일 내부의 순서와 다릅니다.

한글로젠은 단순히 HWP 파일을 읽어서 처리하기 때문에 getAll() 함수는 눈으로 보는 순서와 다를 수 있습니다.

Q3) HWPGenSaver.toBase64() 함수를 실행할 때 java.lang.NoClassDefFoundError 오류가 발생합니다.

Answer) 오라클에서 제공하는 표준 JRE가 아닌 Open JRE는 특정한 Class가 누락될 수 있습니다.

이와 같은 경우에는 클래스가 포함된 아래의 라이브러리를 첨부하여 오류를 해결할 수 있습니다.

Group Id	Artifact Id	Version	License
javax.xml.bind	jaxb-api	2.3.1	CDDL 1.1

Q4) 필드 이름이 동일한 경우 문제가 발생합니다.

Answer) 한글로젠의 기능은 필드 이름을 기준으로 동작합니다.

HWP 파일에서 동일한 필드 이름이 존재하는 경우, 원하지 않는 결과를 도출할 수 있습니다.

HWP 파일에서 필드 이름을 설정할 때 유의하여 주시기 바랍니다.

HANCOM
한컴이노스트림

한글로젠»